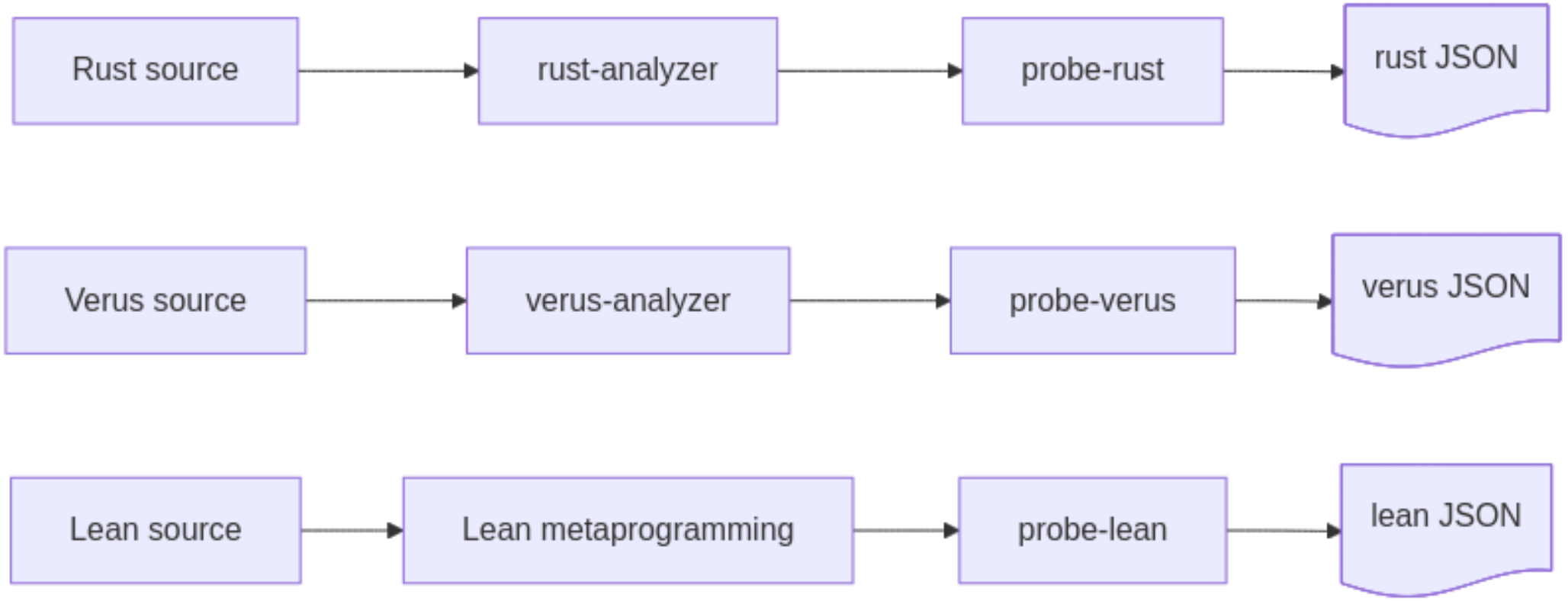
Probes: factual data about (verified) code

A small tool ecosystem that reads what code indexers understand and writes it down as JSON.

What the probes are

Code indexers already parse and understand a codebase. The probes read that understanding and emit **structured, factual data** — no interpretation, no styling.



probe-aeneas has no indexer of its own: it runs probe-rust and probe-lean and *joins* their

The JSON: one entry per code atom

Every probe emits the same shape — one entry per atom (a Rust function, a Verus construct, a Lean construct) with its dependencies and status.

Project	Typical information per atom
Rust	function calls (the call graph)
Verus, Aeneas	calls + verification status; thm deps + proof status
Lean	theorem dependencies + proof status

A Verus atom

A Rust function, its calls, and whether it verifies against its spec.

```
"probe:curve25519-dalek/.../[ProjectivePoint]double()": {
  "kind": "exec",
  "language": "rust",
  "primary-spec": "requires is_valid_projective_point(*self), ensures ...",
  "verification-status": "transitively-verified",
  "dependencies": [
    "probe:.../[FieldElement51]square()",
    "probe:.../[FieldElement51]square2()"
  ]
}
```

probe-aeneas: Rust ↔ Lean, linked

An Aeneas project has two sides: the **Rust crate** and the **Aeneas-generated Lean** that models it plus the specs proved about it.

- **probe-rust** indexes the crate and tags each function with a **Charon-derived qualified name**.
- **probe-lean** indexes the Lean side, where each generated def remembers the Rust function it came from.

Charon names are the **shared vocabulary**: matching them links a Rust function to the Lean def that implements it and the theorem that specifies it.

Where spec and proof live differ by tool

Same property (`FieldElement51::is_negative`), verified both ways.

Verus	Lean via Aeneas
Code, **spec** and **proof** are **one artifact**	Code is a plain `def`; **spec** + **proof** are a **separate theorem**
spec = `requires` / `ensures` contract	**spec** = the theorem's statement
proof = `proof { }` inside the fn	**proof** = the proof term

This split is why "what does a Lean `def` mean?" is harder than in Verus — a `def` can be an impl, a spec, or neither.

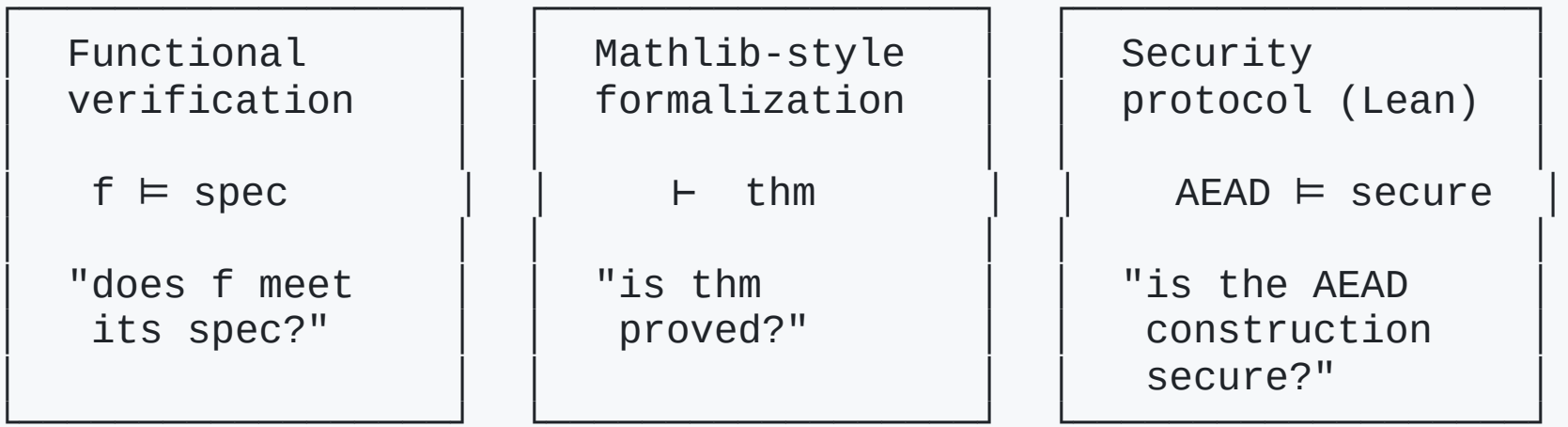
A Lean `def` is multi-faceted

Verus has dedicated syntax (`spec fn` vs `fn`). Lean has only `def` — one keyword for an implementation, a spec, or something else. **The syntax is identical**, so in general we cannot tell a spec-def from an impl-def.

Consequence: **no single colour per `def`** — it depends on the project.

- **Aeneas projects:** for Aeneas-generated translations (they model implementations) they get the colour of the specs, **blue** for defs that play the role of a Verus `spec fn`.
- **Mathlib-style projects:** there is a proposal to have a **green** dot to denote the def "**compiles**", not "meets a spec".
- **Verification projects done in Lean (like lean-zip):** without annotations we cannot distinguish defs representing impls from def representing specs

Three kinds of projects, three questions

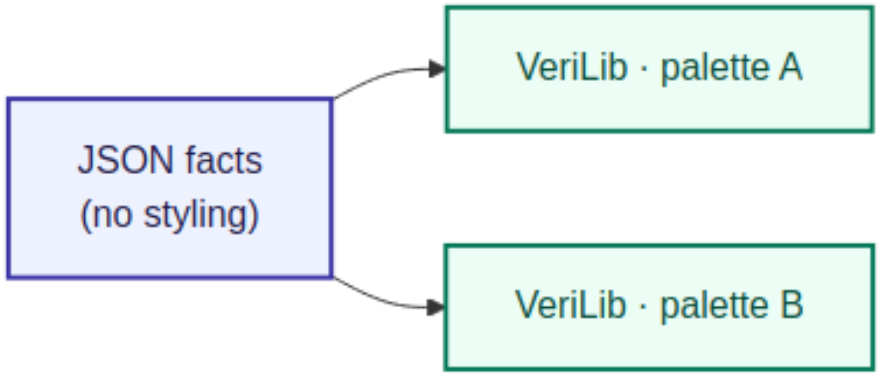


Different questions in nature — forcing all three into one framework loses meaning.

Division of labour

The **probes** have one job: report factual data about the code, as JSON. They take **no position** on how it should look.

VeriLib takes that data and presents it — by colouring atoms and computing statistics.

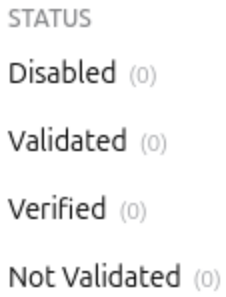


What colours and which stats are **subjective**: helpful to one user, "nay" to another. So the colour/status choices below are a VeriLib concern to reach consensus on — not facts the probes emit.

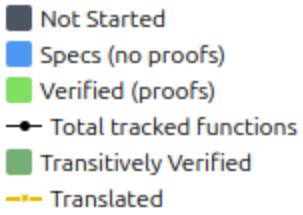
Currently

In VeriLib we see:

Statuses:



Colors:



There's a gap between them (we should have a 1-to-1 correspondence between statuses and colours)

Statuses

The probes don't (cannot) emit info about:

- a function being tracked
- a spec being validated

In the current verif projects:

- specs are validated through PRs, if a spec exists in the codebase, then it's validated
- there's nothing in the code saying that a function is tracked

Colour ↔ status proposal

A one-to-one mapping between verification status and colour:

- **disabled** — no spec / outside verification scope
- **translated** — Aeneas def with no spec yet
- **sorry / assumes**
- **error**
- **verified** — a function meets its spec, or a theorem is proved
- **trusted** — axioms / assumed

Shapes

Colours for statuses, shapes for roles:

- can be def/thm like in verso-blueprint; not sure if it makes sense for verus…
- can be impl/spec/proof; could make sense for Verus; not much for Lean spec-theorems; maybe just impl/spec? just have a shape for specs?

What we have in the jsons:

- kinds: def/abbrev/thm/… for lean; exec/spec/proof for verus

Arrows

The following fields in the jsons from the probes can allow us to have different arrows:

- `translation-name`
- `primary-spec`
- `dependencies`

Open question 1 — green

Should green mean *only* "satisfies its spec / is proved"?

- If yes, **arbitrary Lean `def`s get no verification colour** ("this def compiles" is diff from "this function meets its spec").
- Aeneas-generated defs still get a colour: **translated** until specified, then the colour of their spec.
- for vcvio based projects, we can leverage info like Jin implemented
- for verso-blueprint projects, we have a probe wrapping around verso-blueprint to get all that the user annotated

Open question 1 — resolution

Separate verification of Rust functions from checking of artifacts — two visual channels:

- **Rust exec → colour *bar* only** — the bar carries the *verification* status of the function (does the implementation meet its spec).
- **Verus spec, Verus proof, Lean def, Lean type declaration, Lean axiom, Lean theorem → colour *dot* only** — the dot carries the *checking* status of the artifact (does it go through).

The dot colour reflects whether the tool accepts the artifact:

- Lean atoms → `lake build` succeeds or not.
- Verus atoms → `cargo-verus verify` succeeds or not.

This keeps "the function meets its spec" (bar) distinct from "the artifact checks / goes through" (dot).

Open question 2 — blue for specs

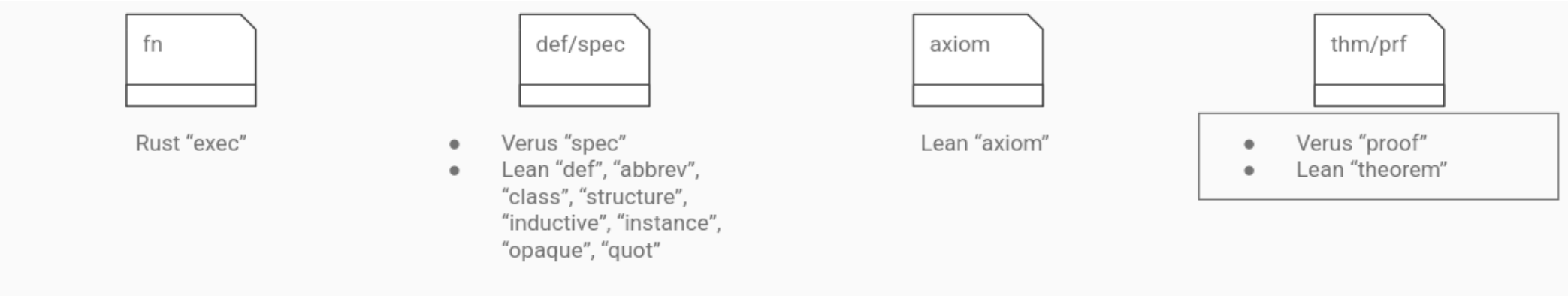
Do we want blue for specs (the spec itself, not a specified function)?

- If a function has a spec, it also has a proof → the outcome is already **green**/**orange**/**red**. **Blue** is "eaten".
- Only genuine case: **spec defs** used in pre/post-conditions (Verus `spec fn`, similar Aeneas defs).

Or we could see a spec as a **role** → maybe distinguish specs by **shape**, not colour.

Open question 2 — resolution

No colour for specs. A spec is a **role**, read off the atom's `kind` in the JSON, and shown by **shape** — colour stays reserved for status.



Shape	Kinds
`fn`	Rust `exec`
`def/spec`	Verus `spec`; Lean `def`, `abbrev`, `class`, `structure`, `inductive`, `instance`, `opaque`, `quot`
`axiom`	Lean `axiom`
`thm/prf`	Verus `proof`; Lean `theorem`

Open question 3 — tracking

Is every Rust function "tracked" by default?

- **Yes** → a finished project needs an `outside_verif_scope` annotation, else it shows as incomplete (has an upper bound).
- **No** (current) → a function with no spec is disabled; no upper bound, but the progress chart still grows as specs are added.

Open question 3 — resolution

Yes — every Rust function is tracked by default. Being outside verification scope is stated explicitly in the code, and the probes read it:

- **Verus** → `#[verifier::external]` marks a function as outside verification scope → status **disabled**.
- **Aeneas** → by default every Rust function is translated, hence tracked. A Lean annotation on the translation ("this translation won't be verified") flips that function to **disabled**.

So the upper bound is well-defined: all functions count, minus the ones explicitly annotated as out of scope.

Open question 4 — pure Rust projects

Should VeriLib display pure Rust (unverified) projects at all?

- If yes, what colour are those atoms?
- white (as neutral) conveys "not for verification" — but today white is *also* used for tracked functions, which is inconsistent.

Open question 4 — resolution

Yes — display them, in white. The earlier inconsistency disappears once verification lives on the **colour bar** (Open question 1):

- Pure Rust functions → white, no bar.
- Verified Rust functions → the **bar** carries their status.

So white is no longer overloaded: a verified function is told apart from a pure-Rust-project function by its colour bar, not by its fill.

Open question 5 — "done"

When is a project done?

- Verification: when Rust functions are **green**?
- Formalization: when theorems are **green**?
- Without a notion of tracking, **any intermediate state looks "done"**.

Open question 5 — resolution

Solved by Open question 3's default. Since every Rust function is to-be-verified by default (minus the explicitly out-of-scope ones), the denominator is fixed up front:

- **Done** = all tracked functions are **green**.
- Intermediate states no longer look "done": untouched functions still count against the upper bound, so the progress chart only reaches 100% when the whole tracked set is verified.

Open question 6 — trusted / axioms

Distinguish axioms by colour (purple) or by shape?

- Colour lets us **quantify** how much is trusted (how much purple).
- Shape does not quantify as easily.

Open question 6 — resolution

Colour: purple for trusted atoms. Since trust is a status, it goes on the colour channel — letting us quantify how much of a project rests on axioms / assumptions (how much purple), which shape alone wouldn't give us.